“架构辩论赛”记录

- 注：因为AI回答较长，将先对辩论结果进行总结，后续会提供完整的对话记录。

结果分析与总结

经过对AI的多轮提问和分析，最后我组选择了单体分层架构（Layered Monolithic Architecture with Component-based design）作为CampusHub校园互助平台的主体架构风格。 同时我们经过对几个AI提供的回答进行汇总分析可以发现，虽然不同的AI在针对我们的项目架构选择中都显然是以单体分层架构为主的，但是也出现了其他一些架构风格作为项目功能的补充， 例如事件驱动架构（Event-Driven Architecture）被推荐用于订单状态机和信用体系的解耦，DDD被推荐用于核心领域的设计。我们最后也对这样的架构组合进一步进行了分析和了解， 最终认为，可以使用单体分层架构为整体，局部使用DDD以及使用事件驱动来进行交互的设计。

方式1-提供详细项目约束后问

使用模型：Gemini

提问文本：上面是我们校园平台项目的需求规格说明书，能否帮我推荐架构风格（将P1中的需求规格说明书提供给AI）

AI 回答

针对 CampusHub 校园互助平台的需求规格说明书，考虑到你们 10 周的开发周期、240 人时的工作量以及前后端分离的明确要求，我为你推荐以下架构方案。考虑到该项目既包含**交易类（高一致性、状态流转）又包含信息类（高频访问、内容聚合）**的特点，最适合的架构风格是：基于 Spring Boot 的分层式微内核单体架构（Layered Monolithic Architecture with Component-based design）。

以下是详细的架构推荐及理由：

1. 逻辑架构：四层分层模型

由于项目需要处理复杂的“订单状态机”和“信用评价体系”，清晰的分层可以确保业务逻辑不被 UI 逻辑污染。

- 接入层（Web Layer / Controller）：负责处理前端 Vue 3 的 RESTful 请求，进行初步的参数校验（如学号格式验证）。
- 业务逻辑层（Service Layer）：
  - 交易域：处理接单、状态流转、双方确认的原子性操作。
  - 信用域：处理评分计算、高信用用户认定逻辑。
- 数据访问层（Repository / DAO）：使用 Spring Data JPA 或 MyBatis-Plus 操作 MySQL。
- 模型层（Domain/Entity Layer）：定义用户、订单、需求等核心实体。

2. 功能模块划分（Component-based）

虽然建议采用单体部署（减小运维压力），但在代码结构上应采用按业务模块拆分的方式，以便于后期向微服务平滑演进：

模块名	核心职责	技术关键点
Auth Module	注册登录、学号校验、JWT 权限控制	密码加密存储、Token 刷新
Trade Module	需求发布、订单状态机控制	状态模式（State Pattern）维护订单流转
Social Module	信息交流区、发帖、点赞	异步处理点赞统计（避免阻塞主流程）
Credit Module	信用分计算、投诉仲裁投票机制	权重算法设计
Admin Module	内容审核、违规处理、看板数据	数据聚合查询

3. 关键技术架构决策

状态机驱动的订单流程
需求 3.1.3 提到的“接单 → 进行中 → 完成 → 评价”是一个典型的状态流转过程。
推荐方案：在后端实现一个轻量级的状态机（State Machine）。这样可以严格控制非法操作（例如：处于“已完成”状态的订单不能再点击“取消”），保证数据的一致性。

信用体系的异步解耦
需求 3.1.4 和 3.1.5 中，评价和纠纷处理会影响信用分。
推荐方案：采用事件驱动思维。当订单完成或纠纷处理结束时，发布一个“信用变更事件”，由专门的信用服务去异步更新分数。这样即使信用分模块出现小问题，也不会影响到核心的互助交易流程。

扩展性与 AI 协同
前后端分离方案：前端使用 Vue 3 + Pinia（状态管理），后端提供 OpenAPI（Swagger）文档。
AI 辅助点：由于你们需要记录《AI 协作反思日志》，建议使用 GitHub Copilot 或 Cursor。架构上保持 Controller 层逻辑尽可能薄，让 AI 能够精准生成 Service 层的业务代码。

4. 部署架构（MVP 阶段）

由于周期仅 10 周，不建议过度设计（如引入 Redis 集群或 Kafka）。

- 前端：静态资源部署（Nginx）。
- 后端：单个 JAR 包运行。
- 数据库：单实例 MySQL。
- 文件存储：头像及证据图片可直接存储在云对象存储（如 OSS/COS）或简单的本地映射。

5. 为什么不推荐完全的微服务？

虽然微服务很火，但针对你们的情况：

- 开发成本：240 人时非常紧张，微服务带来的服务发现、链路追踪、分布式事务会占据 30% 以上的额外时间。
- 团队协作：校园项目成员间沟通成本低，单体架构配合良好的 Git 分支管理（Git Flow）效率最高。
- 性能需求：需求 3.2.1 要求加载小于 2 秒，单体架构由于没有网络开销，在小规模并发下性能反而更优。

建议：采用**“大单体、小模块”**的开发模式，既能保证交付进度，又能满足软件工程作业对系统设计严谨性的要求。

方式2-提供简单描述直接询问

使用模型：ChatGPT

提问文本：我们要开发一个校园互助平台，用户可以发布需求、接单、评价对方。请推荐适合的架构风格，并说明理由。

AI 回答

你这个场景本质上是一个典型的双边平台（two-sided marketplace）：发布需求的一方 + 接单的一方 + 信任体系（评价/信用）。

先说结论：不要一上来就微服务，大多数团队都会在这里踩坑。

一、推荐架构（分阶段）

✅ 阶段 1：模块化单体（强烈推荐起步方案）

架构风格：模块化单体（Modular Monolith） + 分层架构

核心模块划分：

- 用户模块（注册 / 登录 / 认证）
- 需求发布模块（发布 / 编辑 / 下架）
- 订单模块（接单 / 状态流转）
- 评价模块（评分 / 评论）
- 通知模块（消息 / 推送）
- 权限与风控模块（防刷 / 举报）

技术结构：

- 表现层（Controller）
- 业务层（Service）
- 数据层（Repository / DAO）

👉 关键点：模块之间用接口隔离，而不是直接互相调用数据库。

👍 为什么推荐这个？

1. 业务还在快速变化
   校园平台初期：
   - 功能会频繁改（比如：要不要押金？要不要聊天？要不要竞标？）

    - 数据模型不稳定

    👉 微服务在这种阶段会让你：

    - 改一个字段要改多个服务
    - 接口联调成本爆炸

2. 团队规模通常不大
   如果你们是 1～5 人开发，微服务会带来：

    - 服务拆分成本
    - 部署复杂度（Docker / K8s）
    - 分布式问题（事务 / 一致性 / 调试）

    👉 收益远小于成本。

3. 性能瓶颈不在架构
   校园项目初期瓶颈通常是：

    - SQL 写得差
    - 没有缓存
    - 并发控制不好

    而不是“单体不够扩展”。

二、建议的技术架构

后端

- Java：Spring Boot
- 或 Python：Django / FastAPI

👉 推荐 Spring Boot（生态成熟 + 易扩展）。

前端

- Web：Vue.js / React
- 移动端：小程序（校园很适合）或 Flutter

数据层

- 主数据库：MySQL
- 缓存：Redis

可选增强

- 搜索：Elasticsearch（做需求搜索）
- 消息队列：RabbitMQ（通知、异步任务）

三、核心设计（比架构更重要）

1. 订单状态机（重点）
   例如：

   ```
   已发布 → 已接单 → 进行中 → 已完成 → 已评价
                  ↓
                 已取消
   ```

   👉 状态流转必须严格控制（否则会出现“重复接单”）。

2. 防并发问题
   典型问题：多人同时抢单。
   解决：

   - 乐观锁（version）
   - Redis 分布式锁

3. 信任体系（平台成败关键）

   - 评分系统（1 ~ 5 星）
   - 历史记录不可篡改
   - 举报 / 仲裁机制

4. 权限模型

   - 普通用户
   - 接单者（可升级）
   - 管理员

四、什么时候升级为微服务？

当出现这些情况再考虑拆：

- 用户量 > 10 万
- 某些模块压力特别大（比如消息 / 搜索）

- 团队 ≥ 10 人
- 部署频率非常高（需要独立发布）

👉 可以逐步拆：通知服务、搜索服务、用户服务，而不是“一刀切”。

五、一个现实建议（很重要）

很多校园项目失败的原因不是技术，而是：

- ❌ 没人用
- ❌ 冷启动困难
- ❌ 供需不平衡

所以你应该优先考虑：

- 如何获取第一批用户
- 如何保证需求能被接单
- 是否需要“官方托底”（比如平台自己接单）

总结

最合适的路线是：模块化单体 →（验证成功）→ 按需拆微服务。

而不是：❌ 一上来就微服务（99% 会翻车）。

方式3-让 AI对比多种方案的优劣

使用模型：Deepseek

提问文本：我们要开发一个校园互助平台，用户可以发布需求、接单、评价对方。请对比一些常用的架构风格，并推荐最适合我们的。

AI 回答

针对 CampusHub 校园互助平台的开发需求，我们对比几种常用架构风格，并结合项目规模、团队时间、技术约束等因素，给出推荐。

常用架构风格对比

架构风格	核心思想	优点	缺点	适用场景
单体分层架构	将应用划分为表示层、业务层、数据访问层，部署为单一进程	开发简单、调试方便、部署单一、事务管理容易；适合小型团队快速迭代	随着功能膨胀，代码耦合度上升；部分模块改动需全量测试和部署	中小规模、团队人数少、开发周期短的项目
微服务架构	按业务能力拆分为多个独立服务，每个服务独立部署和数据库	服务间解耦、独立扩展、技术异构、故障隔离	运维复杂（服务发现、配置中心、链路追踪）；分布式事务困难；初期开发成本高	大型系统、多团队协作、需要高可扩展性的场景
客户端-服务器 (C/S) + 前后端分离	前端（Vue）和后端（Spring Boot）独立开发，通过 HTTP/RESTful API 通信	职责清晰、前端可独立部署、后端可复用、支持多端（Web/App）	需要在接口设计上花费额外精力；跨域、鉴权等需额外处理	现代 Web 应用，尤其需要多端支持或前端动态化
事件驱动架构	组件间通过异步事件（消息队列）通信，解耦生产者和消费者	高吞吐、松耦合、弹性伸缩	架构复杂、难以追踪数据流、事件一致性保证困难	高并发、实时流处理、异步长任务场景
RESTful 架构	资源导向，使用 HTTP 方法表达操作，无状态通信	简单、易于缓存、扩展性好、与 Web 基础设施天然集成	对于复杂事务支持较弱；过度使用可能造成过多网络往返	公开 API、前后端分离、需要被多种客户端使用

针对 CampusHub 的分析

1. **项目规模和周期**：10 周开发，团队总计约 240 人时（约 3-4 人）。功能以用户管理、需求发布、订单流程、评价信用为主，模块间耦合明确（订单流程依赖用

户和需求数据），没有独立扩缩容或异构技术栈的必要性。**微服务和事件驱动严重过重，会显著增加工期风险。**

2. **技术约束**：需求明确要求前后端分离（Spring Boot + Vue 3）。这意味着自然采用 **客户端-服务器风格**，后端提供 API，前端单独部署。

3. **后端内部组织**：后端虽然作为单一服务，但内部应按职责分层：Controller（表现层）、Service（业务层）、Repository（数据访问层）。这符合 **分层架构**，便于单元测试和维护。

4. **性能与扩展性**：校园场景用户规模有限，单体分层架构 + 合理的数据库索引足以满足“首页加载 <2 秒”的要求。未来如需扩展，可先水平复制后端实例（无状态设计），再按需拆分为微服务。

5. **团队协作**：分层架构配合明确的 API 契约（如 Swagger），前后端可并行开发；而微服务会增加服务间接口定义、服务注册等额外沟通成本。

推荐架构方案

分层架构（后端） + 客户端-服务器（前后端分离）+ RESTful API

- **总体结构**：
  - **前端**：Vue 3 单页应用，负责页面渲染、用户交互。
  - **后端**：Spring Boot 单体应用，划分为三层：
    - Controller 层：接收 HTTP 请求，参数校验，返回 JSON。
    - Service 层：业务逻辑（订单状态机、信用计算、纠纷处理）。
    - Repository 层：通过 JPA/MyBatis 访问 MySQL。
  - **数据库**：MySQL 存储用户、需求、订单、评价、投诉等数据。

- **通信方式**：HTTP + RESTful API（如 `POST /api/orders`、`PATCH /api/orders/{id}/status`）。无状态设计，使用 JWT 或 Session 管理登录态。

- **核心优势**：
  - 符合课程作业对前后端分离的建议。
  - 开发效率高：Spring Boot + Vue 社区成熟，大量模板可用。
  - 易于测试：分层使单元测试和集成测试边界清晰。
  - 满足非功能需求：可扩展（新增需求分类只需加数据表和枚举），可维护（代码分层清晰），性能足够（可增加缓存层）。

- **放弃微服务的理由**：
    - 项目规模不足，引入微服务的运维负担（服务发现、配置中心、API 网关）远超收益。
    - 订单、用户、信用等模块天然需要强一致性，单体的事务支持更简单。

补充建议

- **API 设计**：遵循 RESTful 规范，使用名词复数 + HTTP 方法，状态码语义化（如 201 Created、400 Bad Request）。
- **部署**：前端打包后可由 Nginx 托管，后端打为 jar 包运行，简单可靠。
- **未来演进**：若用户量激增，可先将后端 Session 抽取到 Redis（无状态化），再按功能拆分（如订单服务、用户服务）过渡到微服务，但当前阶段无须考虑。

综上，**分层架构（单体后端） + 前后端分离（RESTful API）** 是 CampusHub 最合适的架构风格，在开发效率、可维护性和性能之间取得了最佳平衡。

方式4-让 AI 扮演架构师角色进行分析

使用模型：ChatGPT

提问文本：你是一个资深软件架构师，我们要开发一个校园互助平台，用户可以发布需求、接单、评价对方。请分析这个项目的特点，并推荐最适合的架构风格。

AI 回答

我先直接说结论：这个项目最适合采用“分层单体 + 领域驱动设计（DDD Lite）+ 事件驱动增强”的架构风格，而不是一上来就微服务。原因和分析我给你拆清楚。

一、项目本质特征分析（这是选架构的关键）

结合你这份需求文档，这个系统有几个非常典型的特征：

1️⃣ 强业务流程驱动（核心是“交易闭环”）

平台的核心不是信息展示，而是订单状态流转：

发布 → 接单 → 进行中 → 完成 → 评价 → 信用沉淀

这意味着：

- 存在明确的状态机
- 有一致性要求（不能乱跳状态）
- 涉及多角色协作（需求方 + 服务方）

👉 本质：事务型系统（类似轻量版“闲鱼 + 滴滴”）。

2️⃣ 多子域但强耦合

系统模块：

- 用户系统
- 需求发布
- 订单系统（核心）
- 评价/信用系统
- 消息系统
- 管理后台

这些模块的特点：

👉 围绕“订单”高度耦合。

比如：

- 评价依赖订单完成
- 信用依赖评价
- 消息依赖订单状态变化

👉 不适合一开始拆微服务（否则会被分布式事务折磨）。

3️⃣ 用户规模有限（校园场景）

文档中明确：校园内用户规模有限，分阶段迭代。

意味着：

- QPS 不高
- 数据量可控
- 单机完全可撑住

👉 没有微服务的性能动机。

4️⃣ 开发资源非常有限

- 10 周
- 240 人时

👉 现实点说：做微服务 = 项目直接失败概率上升。

因为：

- 运维复杂（部署、网关、服务发现）
- 调试困难
- 学生成本高

5️⃣ 需要一定扩展性（但不是现在）

非功能需求提到：

- 可扩展分类
- 增加评价维度
- 增加消息类型

👉 是“未来扩展”，不是“现在高并发”。

二、架构选择对比（帮你排坑）

❌ 微服务架构（不推荐）

问题：

- 分布式事务（订单+评价+信用）
- 服务拆分困难（边界不清）
- 运维成本高
- 调试困难

👉 对这个项目来说：过度设计。

❌ 传统三层架构（不够）

Controller → Service → DAO

问题：

- 业务复杂后 Service 会变成“上帝类”
- 订单状态逻辑容易混乱
- 扩展困难

👉 不够支撑“交易闭环系统”。

✅ 推荐：分层单体 + DDD Lite + 事件驱动

这是最合适的组合。

三、推荐架构方案（重点）

🧱 1. 总体架构

前端（Vue3）↓ API层（Controller）↓ 应用层（Application Service）↓ 领域层（Domain）↓ 基础设施层（Repository / DB / MQ）

🧠 2. 核心思想：DDD Lite（轻量领域驱动）

不用完整 DDD，只抓核心：

核心领域划分（非常关键）

① 用户域（User）

- 用户信息
- 身份认证
- 状态（封禁等）

② 需求域（Task / Request）

- 发布需求
- 浏览/筛选

③ 订单域（Order）⭐ 核心

- 状态机管理
- 接单逻辑
- 完成确认

👉 这是系统最重要的聚合根。

④ 评价与信用域（Review / Credit）

- 双向评价
- 信用计算

⑤ 消息域（Message）

- 通知
- IM（后期）

👉 每个域：

- 有自己的 Entity / Service
- 不直接跨域操作数据库

🔄 3. 引入“事件驱动”（关键加分项）

为了降低耦合，引入简单事件机制：

例如：

场景：订单完成

订单完成 ↓ 发布事件：OrderCompletedEvent ↓ 触发：

- 评价模块
- 信用计算
- 消息通知

👉 实现方式：

- 简单版：Spring 事件机制
- 进阶版：消息队列（如 RabbitMQ）

🧩 4. 订单状态机（必须设计）

这是系统核心，建议明确建模：

CREATED → ACCEPTED → IN_PROGRESS → COMPLETED → REVIEWED

用：

- 枚举 + 状态校验
- 或状态模式（进阶）

👉 防止：

- 跳状态
- 并发错误

🗄️ 5. 数据架构建议

- MySQL（主库）
- Redis（缓存 + 登录态）
- 可选：MQ（后期）

🚀 6. MVP 阶段架构（非常现实）

你们现在只需要：

- 单体 Spring Boot
- 分层 + 模块化
- 不拆微服务
- 不上 MQ（先用同步或本地事件）

👉 10 周一定能做完。

四、为什么这个架构“最合适”

总结一下本质原因：

维度	结论
业务复杂度	中等（有交易闭环）
并发规模	低
开发资源	极少
扩展需求	有，但不紧急

👉 最优解：用单体保证开发效率，用 DDD 保证结构清晰，用事件机制保证可扩展。

五、给你的“架构演进路线”（很重要）

第一步（现在）

- 单体 + 分层 + DDD Lite

第二步（课程后）

- 引入 Redis
- 引入 MQ（解耦消息/评价）

第三步（用户增长）

拆微服务：

- 用户服务
- 订单服务
- 消息服务

👉 这才是正确路径（而不是一开始就微服务）。